Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase III · Secuencias y Transformer

Módulo 13

Atención

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 13: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
n, d	longitud de la secuencia y dimensión del modelo
$x_i \in \mathbb{R}^{1\times d}$, $X \in \mathbb{R}^{n\times d}$	vector (fila) del token i y matriz que los apila
$q_i, k_j \in \mathbb{R}^{1\times d_k}$, $v_j \in \mathbb{R}^{1\times d_v}$	consulta (*query*), clave (*key*) y valor (*value*)
Q, K, V	matrices que apilan consultas, claves y valores
$W_Q, W_K \in \mathbb{R}^{d\times d_k}$, $W_V \in \mathbb{R}^{d\times d_v}$	matrices de proyección aprendidas
$S = QK^\top$, $s_{ij} = q_i \cdot k_j$	matriz de puntuaciones (parecidos)
A, α_{ij}	matriz de atención y peso que la posición i asigna a la j
$O = AV$, o_i	salida de la atención
M	máscara aditiva, con entradas 0 o $-\infty$
h, W_O	número de cabezas y proyección de salida
β, $\tau = 1/\beta$	inverso de la temperatura y temperatura
P	matriz de permutación
$\mathrm{conv}\{v_1, \ldots, v_n\}$	envolvente convexa de los valores
e_t, p_i	vectores *one-hot* de token y de posición (tema 5)

Los vectores de token, consulta, clave y valor son vectores fila, como en [11] y [8]; así, $Q = XW_Q$. La softmax de una matriz se aplica por filas.

Índice

Sobre este resumen **ii**

1 Atención como agrupación ponderada **1**
1.1 Del diccionario a la búsqueda suave . . . 1
1.2 Propiedades de la agrupación . . . 2
1.3 Regresión por núcleos de Nadaraya y Watson . . . 2
1.4 La atención aditiva de Bahdanau . . . 3

2 Atención por producto escalar escalado **4**
2.1 Forma matricial . . . 4
2.2 Equivariancia por permutaciones . . . 4
2.3 Por qué se divide por $\sqrt{d_k}$ . . . 5

3 La máscara causal **7**
3.1 Máscaras aditivas . . . 7
3.2 Causalidad . . . 8

4 Atención multi-cabeza y coste computacional **9**
4.1 Definición . . . 9
4.2 Parámetros y estructura . . . 9
4.3 Coste en operaciones . . . 10
4.4 Memoria y contexto largo . . . 10

5 Circuitos de atención y memorias asociativas **12**
5.1 El flujo residual y los circuitos QK y OV . . . 12
5.2 Cabezas de inducción . . . 12
5.3 Atención y redes de Hopfield modernas . . . 13

Bibliografía **15**

Tema 1

Atención como agrupación ponderada

Este tema define la atención en su forma más general: una consulta se compara con un conjunto de claves, los parecidos se convierten en pesos mediante una softmax y la salida es la media de los valores asociados ponderada por esos pesos. Se demuestran las dos propiedades estructurales básicas de la operación, la pertenencia de la salida a la envolvente convexa de los valores y su comportamiento en los límites de temperatura, y se presentan sus dos antecedentes: la regresión por núcleos de Nadaraya y Watson y la atención aditiva de Bahdanau, Cho y Bengio. Se siguen las §11.1, §11.2 y §11.4 de [12], la §3.1 de [1], la §7.1 de [8] y la clase 5 de [3].

1.1 Del diccionario a la búsqueda suave

Un diccionario almacena pares (k_j, v_j) y, ante una consulta q, devuelve el valor v_j cuya clave coincide con q, o un error si ninguna coincide. La búsqueda es *dura*: el resultado depende de q de forma discontinua y no admite gradientes. La atención sustituye la coincidencia exacta por un grado de parecido y la elección de un valor por una media ponderada [12, §11.1].

Definición 1.1 (Agrupación por atención). Sean $q \in \mathbb{R}^{1\times d_k}$ una *consulta*, $k_1, \dots, k_n \in \mathbb{R}^{1\times d_k}$ unas *claves* y $v_1, \dots, v_n \in \mathbb{R}^{1\times d_v}$ los *valores* asociados. Sea $a\colon \mathbb{R}^{1\times d_k} \times \mathbb{R}^{1\times d_k} \to \mathbb{R}$ una *función de puntuación*. La *agrupación por atención* de los valores respecto de q es

$$\operatorname{Attn}\big(q; (k_j, v_j)_{j=1}^n\big) = \sum_{j=1}^{n} \alpha_j\, v_j, \qquad \alpha_j = \frac{\exp a(q, k_j)}{\sum_{l=1}^{n} \exp a(q, k_l)},$$

es decir, $(\alpha_1, \dots, \alpha_n) = \operatorname{softmax}\big(a(q, k_1), \dots, a(q, k_n)\big)$. Los números α_j se llaman *pesos de atención*.

La función de puntuación más usada es el producto escalar, $a(q, k) = q \cdot k = qk^\top$, que es grande cuando los dos vectores apuntan en direcciones parecidas y tienen norma grande (módulo 2). Las claves deciden *cuánto* contribuye cada par; los valores, *qué* se contribuye. Separar ambos papeles permite que el criterio por el que un elemento es encontrado no coincida con la información que entrega.

Definición 1.2 (Autoatención). Sean $x_1, \dots, x_n \in \mathbb{R}^{1\times d}$ los vectores de los tokens de una secuencia y $W_Q, W_K \in \mathbb{R}^{d\times d_k}$, $W_V \in \mathbb{R}^{d\times d_v}$ matrices de parámetros. La *autoatención* asigna a cada posición i la salida $o_i = \operatorname{Attn}\big(q_i; (k_j, v_j)_{j=1}^n\big)$, con

$$q_i = x_i W_Q, \qquad k_j = x_j W_K, \qquad v_j = x_j W_V.$$

Si las consultas proceden de una secuencia y las claves y valores de otra, se habla de *atención cruzada*.

Observación 1.3. La figura 1.1 resume la definición. En un modelo de lenguaje del tipo *decoder-only*, como los que se estudian en este curso, toda la atención es autoatención; la atención cruzada aparece en los modelos codificador-decodificador de traducción (apartado 1.4).

1.2 Propiedades de la agrupación

Proposición 1.4 (Envolvente convexa). *Para cualesquiera q, claves, valores y función de puntuación, la salida de la definición 1.1 pertenece a la envolvente convexa* $\operatorname{conv}\{v_1, \dots, v_n\}$. *En particular, para cada coordenada c,*

$$\min_j v_j[c] \;\le\; \operatorname{Attn}(q)[c] \;\le\; \max_j v_j[c].$$

La atención, por tanto, solo *mezcla* información existente: no puede producir una coordenada mayor que la mayor de los valores. Por esa razón un bloque Transformer combina la atención con una red de alimentación hacia delante y con conexiones residuales (módulo 14).

Para estudiar la dependencia respecto de la escala de las puntuaciones se introduce un factor $\beta > 0$, el *inverso de la temperatura* del módulo 1: $\alpha_j(\beta) = \exp(\beta s_j)/\sum_l \exp(\beta s_l)$, con $s_j = a(q, k_j)$.

Teorema 1.5 (Límites de temperatura). *Sean $s_1, \dots, s_n$ las puntuaciones y supóngase que el máximo $s^* = \max_j s_j$ se alcanza en un único índice j^*. Entonces*

$$\lim_{\beta \to +\infty} \operatorname{Attn}_\beta(q) = v_{j^*}, \qquad \lim_{\beta \to 0^+} \operatorname{Attn}_\beta(q) = \frac{1}{n}\sum_{j=1}^{n} v_j.$$

La atención interpola, pues, entre el diccionario exacto (límite $\beta \to \infty$, que devuelve el valor de la clave más parecida) y la media simple de los valores ($\beta \to 0$). El caso con empates se trata en el ejercicio 1.5.

1.3 Regresión por núcleos de Nadaraya y Watson

La idea de promediar respuestas con pesos que dependen del parecido entre la consulta y unos puntos de referencia es anterior al aprendizaje profundo. Zhang *et al.* [12, §11.2] la presentan a partir de la regresión no paramétrica.

Definición 1.6 (Estimador de Nadaraya–Watson). Sean $(k_j, v_j)_{j=1}^n$ observaciones de entrada y salida y $\kappa(q, k) \ge 0$ un *núcleo*. El estimador de Nadaraya–Watson en el punto q es

$$\hat{f}(q) = \sum_{j=1}^{n} \frac{\kappa(q, k_j)}{\sum_l \kappa(q, k_l)}\, v_j.$$

Con el núcleo gaussiano, $\kappa(q, k) = \exp\!\big(-\tfrac{1}{2}\|q - k\|^2\big)$.

Proposición 1.7. *Con el núcleo gaussiano, el estimador de Nadaraya–Watson es la agrupación por atención con puntuación $a(q, k) = -\tfrac{1}{2}\|q - k\|^2$. Si además todas las claves tienen la misma norma, coincide con la agrupación con puntuación $a(q, k) = q \cdot k$.*

1.4 La atención aditiva de Bahdanau

En un traductor codificador-decodificador con redes recurrentes, el codificador produce un vector h_j por cada palabra de la frase de origen. En el modelo clásico, el decodificador solo recibía el último de ellos, un resumen de tamaño fijo de toda la frase; Bahdanau, Cho y Bengio propusieron que en cada paso el decodificador construyese su propio resumen como media ponderada de todos los h_j [1, §3.1].

Definición 1.8 (Atención aditiva). Sean $h_1, \dots, h_T$ los vectores del codificador y s_{i-1} el estado del decodificador antes de emitir la palabra i. El *vector de contexto* es

$$c_i = \sum_{j=1}^{T} \alpha_{ij} h_j, \qquad \alpha_{ij} = \frac{\exp e_{ij}}{\sum_{l} \exp e_{il}}, \qquad e_{ij} = v_a^{\top} \tanh\bigl(W_a s_{i-1} + U_a h_j\bigr),$$

donde W_a, U_a y v_a son parámetros aprendidos (los vectores de esta definición son columnas, como en el artículo original).

Es una agrupación por atención en la que la consulta es s_{i-1}, claves y valores son los h_j y la puntuación es una red de una capa oculta [1, apéndice A.1.2]. Vaswani *et al.* señalan que la puntuación por producto escalar tiene una complejidad teórica similar, pero es mucho más rápida en la práctica porque se reduce a productos de matrices optimizados [11, §3.2.1]; es la que se estudia en el resto del módulo.

Tema 2

Atención por producto escalar escalado

Este tema escribe la autoatención de todas las posiciones a la vez como dos productos de matrices y una softmax por filas, resuelve a mano un ejemplo completo, demuestra que la operación es equivariante por permutaciones y justifica el factor $1/\sqrt{d_k}$ con un argumento de varianza y con la jacobiana de la softmax. Se siguen la §3.2.1 de [11], las §7.1.1 y §7.3 de [8], la §11.3 de [12] y la §12.1 de [2].

2.1 Forma matricial

Se apilan las consultas, claves y valores de las n posiciones como filas de $Q = XW_Q \in \mathbb{R}^{n\times d_k}$, $K = XW_K \in \mathbb{R}^{n\times d_k}$ y $V = XW_V \in \mathbb{R}^{n\times d_v}$.

Definición 2.1 (Atención por producto escalar escalado). Para $Q \in \mathbb{R}^{n\times d_k}$, $K \in \mathbb{R}^{m\times d_k}$ y $V \in \mathbb{R}^{m\times d_v}$,

$$\operatorname{Attn}(Q,K,V) = \operatorname{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)V,$$

donde la softmax se aplica a cada fila. La matriz $A = \operatorname{softmax}(QK^\top/\sqrt{d_k}) \in \mathbb{R}^{n\times m}$ se llama *matriz* (o *mapa*) *de atención*.

La entrada (i,j) de $S = QK^\top$ es $q_i \cdot k_j$, de modo que la fila i de A contiene los pesos de la definición 1.1 para la consulta q_i con puntuación $q\cdot k/\sqrt{d_k}$, y la fila i de AV es la salida o_i. La cuadro 2.1 resume las formas. Vaswani *et al.* introdujeron el nombre y la fórmula [11, §3.2.1, ec. (1)]; la tarea 1 de CS336 la adopta con $m \neq n$ para cubrir la atención cruzada [4, §3.4.4].

Paso	Operación	Forma	Significado
1	$S = QK^\top$	$n \times m$	$s_{ij} = q_i \cdot k_j$
2	$S/\sqrt{d_k}$	$n \times m$	escalado (apartado 2.3)
3	$A = \operatorname{softmax}(S/\sqrt{d_k})$	$n \times m$	cada fila es una distribución
4	$O = AV$	$n \times d_v$	fila i: media ponderada de valores

Cuadro 2.1: Formas de las matrices intermedias de la atención.

2.2 Equivariancia por permutaciones

Proposición 2.3 (Equivariancia). *Sea* $f(X) = \operatorname{softmax}\big(XW_Q(XW_K)^\top/\sqrt{d_k}\big)XW_V$ *la autoatención sin máscara y sea* $P \in \mathbb{R}^{n\times n}$ *una matriz de permutación. Entonces* $f(PX) = P\,f(X)$.

La autoatención no distingue, por sí sola, el orden de los tokens: «el perro muerde al hombre» y «el hombre muerde al perro» producen las mismas salidas, solo que en otras filas. Por eso los modelos de lenguaje añaden información de posición a los vectores de entrada (módulo 14), y por eso la máscara causal del tema 3, que sí depende del orden, rompe esta simetría.

2.3 Por qué se divide por $\sqrt{d_k}$

Vaswani *et al.* justifican el factor con un argumento de varianza que enuncian en una nota al pie [11, §3.2.1, nota 4]; Zhang *et al.* [12, §11.3.3] y Jurafsky y Martin [8, §7.1.1] lo reproducen. Se da aquí con demostración.

Teorema 2.4 (Varianza del producto escalar). *Sean $q = (q_1, \dots, q_{d_k})$ y $k = (k_1, \dots, k_{d_k})$ vectores aleatorios cuyas $2d_k$ componentes son independientes, de media 0 y varianza 1. Entonces*

$$\mathbb{E}[q \cdot k] = 0, \qquad \operatorname{Var}(q \cdot k) = d_k, \qquad \operatorname{Var}\Big(\frac{q \cdot k}{\sqrt{d_k}}\Big) = 1.$$

Por el teorema central del límite, $q \cdot k / \sqrt{d_k}$ es aproximadamente $\mathcal{N}(0,1)$ para d_k grande. Sin el factor, con $d_k = 64$ los parecidos tendrían desviación típica 8 en la inicialización y diferencias de 16 entre dos claves serían frecuentes. Una simulación con $2 \cdot 10^5$ pares de vectores gaussianos da varianzas empíricas 4,00 para $d_k = 4$ y 63,8 para $d_k = 64$.

Para ver por qué importa la escala se necesita la jacobiana de la softmax, calculada en el módulo 4 con la regla del cociente.

Proposición 2.5 (Saturación de la softmax). *Sea $p = \operatorname{softmax}(z) \in \mathbb{R}^n$ y $J = \partial p / \partial z$, de entradas $J_{ij} = p_i(\delta_{ij} - p_j)$. Entonces*

$$\max_{i,j} |J_{ij}| \;\le\; 1 - \max_{l} p_l.$$

En consecuencia, si una probabilidad tiende a 1, todas las derivadas parciales de la softmax tienden a 0.

Logits z	$\operatorname{softmax}(z)$	$\max\lvert J_{ij}\rvert$
$(1, 0, 0)$	(0,576; 0,212; 0,212)	0,244
$(8, 0, 0)$	(0,9993; 0,0003; 0,0003)	0,00067

Cuadro 2.2: Con logits ocho veces mayores, las derivadas de la softmax se reducen en un factor de unos 360.

Observación 2.6 (El factor como temperatura). Dividir por $\sqrt{d_k}$ equivale a aplicar la softmax con temperatura $\tau = \sqrt{d_k}$ (módulo 1, tema 4). Sin ella, el modelo empezaría en la región saturada: en el ejemplo 2.2, sin dividir por 2, el peso de la fila 2 sobre la posición 2 pasa de 0,867 a 0,982. Vaswani *et al.* observan, en la misma línea, que sin escalado la atención aditiva supera a la de producto escalar cuando d_k es grande [11, §3.2.1]. Durante el entrenamiento, la red puede modificar la escala efectiva agrandando o encogiendo W_Q y W_K.

Observación 2.7 (Evaluación estable). Como la softmax es invariante al restar una constante a todas las entradas, se evalúa restando el máximo de cada fila antes de exponenciar, lo que

evita desbordamientos en fp16 (módulo 11). La tarea 1 de CS336 exige implementarla así [4, §3.4.4].

Tema 3

La máscara causal

Un modelo de lenguaje autorregresivo factoriza la probabilidad de una secuencia con la regla de la cadena (módulo 1, tema 2): la distribución del token $t+1$ solo puede depender de los tokens $1, \dots, t$. Este tema define las máscaras aditivas, demuestra que sumar $-\infty$ antes de la softmax equivale a restringir la softmax al conjunto de posiciones permitidas, prueba que la máscara causal hace que cada salida dependa solo de su pasado y deduce de ello que una única pasada produce las n predicciones de una secuencia. Se siguen la §3.2.3 de [11], la §7.3 de [8], las §3.4.4 y §3.4.5 de la tarea 1 de CS336 [4] y la §11.3.2.1 de [12].

3.1 Máscaras aditivas

Definición 3.1 (Máscara aditiva). Sea $\mathcal{P} \subseteq \{1, \dots, n\} \times \{1, \dots, m\}$ el conjunto de pares (i, j) *permitidos*, con la condición de que cada fila i contenga al menos un par permitido. La *máscara aditiva* asociada es la matriz $M \in \{0, -\infty\}^{n \times m}$ con $M_{ij} = 0$ si $(i, j) \in \mathcal{P}$ y $M_{ij} = -\infty$ en otro caso. La *atención enmascarada* es

$$\operatorname{Attn}_M(Q, K, V) = \operatorname{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}} + M\Big)V,$$

con el convenio $e^{-\infty} = 0$.

Definición 3.2 (Máscara causal). En la autoatención ($m = n$), la *máscara causal* es la asociada a $\mathcal{P} = \{(i, j) : j \le i\}$:

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

Es triangular estrictamente superior en sus entradas $-\infty$; la diagonal siempre está permitida.

Vaswani *et al.* la describen como el bloqueo de las conexiones «ilegales» de la autoatención del decodificador, fijando a $-\infty$ las entradas correspondientes de la softmax [11, §3.2.3]; Jurafsky y Martin la presentan en la §7.3 de [8], y la tarea 1 de CS336 la implementa con una máscara booleana en la que `True` indica «puede atender» [4, §3.4.4]. La figura 3.1 muestra su estructura.

Proposición 3.3 (Softmax restringida). *Sea $z \in \mathbb{R}^m$ y sea $\mathcal{P}_i \subseteq \{1, \dots, m\}$ no vacío. Si $\mu_j = 0$ para $j \in \mathcal{P}_i$ y $\mu_j = -\infty$ en otro caso, entonces*

$$\operatorname{softmax}(z + \mu)_j = \begin{cases} \dfrac{e^{z_j}}{\sum_{l \in \mathcal{P}_i} e^{z_l}}, & j \in \mathcal{P}_i, \\ 0, & j \notin \mathcal{P}_i. \end{cases}$$

En particular, los pesos permitidos suman 1 y no dependen de las puntuaciones z_j con $j \notin \mathcal{P}_i$.

La condición $\mathcal{P}_i \neq \varnothing$ es necesaria: una fila enteramente enmascarada daría $0/0$, que en coma flotante produce `NaN` (módulo 11). En la máscara causal no puede ocurrir, porque (i, i) siempre está permitido.

Observación 3.4 (Por qué $-\infty$ antes de la softmax). Sumar 0 antes de la softmax no anula nada, porque $e^0 = 1$. Poner a 0 los pesos *después* de la softmax deja filas que suman menos de 1 y pesos del pasado más pequeños de lo debido; hay que renormalizar, y el resultado coincide entonces con el de la proposición 3.3 (ejercicio 3.5). En la práctica se usa `-inf` o un número negativo grande; el ejercicio 3.4 cuantifica cuánto debe valer.

3.2 Causalidad

Teorema 3.6 (Causalidad de la atención enmascarada). *Sean $q_i = x_i W_Q$, $k_j = x_j W_K$, $v_j = x_j W_V$ y sea o_i la fila i de la autoatención con máscara causal. Entonces, para cada i, o_i es función únicamente de $x_1, \ldots, x_i$.*

Corolario 3.7 (Entrenamiento en paralelo). *Sea f la autoatención con máscara causal y, para cada $t \leq n$, sea f_t la misma operación aplicada solo al prefijo $x_{1:t}$. Entonces la fila t de $f(x_{1:n})$ coincide con la última fila de $f_t(x_{1:t})$. En consecuencia, una sola evaluación sobre la secuencia completa proporciona las n representaciones que una red usa para predecir $x_2, \ldots, x_{n+1}$, cada una calculada solo con su prefijo.*

La tarea 1 de CS336 describe la alternativa ingenua, ejecutar la atención n veces, una por prefijo, y la descarta en favor de la máscara [4, §3.4.5]. Una red recurrente necesita, en cambio, n pasos secuenciales, porque el estado en t depende del estado en $t-1$ (módulo 12). Esta diferencia, que Vaswani *et al.* cuantifican como $O(1)$ operaciones secuenciales frente a $O(n)$ [11, §4, tabla 1], es la razón práctica principal por la que los Transformer desplazaron a las redes recurrentes en el preentrenamiento de modelos de lenguaje.

Observación 3.8 (Otras máscaras). La definición 3.1 admite otros conjuntos $\mathcal{P}$. La máscara de relleno (*padding*) prohíbe las posiciones añadidas para igualar la longitud de las secuencias de un lote [12, §11.3.2.1]; la de ventana deslizante permite solo $i - w < j \leq i$, lo que reduce el coste a $O(nw)$ (tema 4). La combinación de varias máscaras es la intersección de sus conjuntos permitidos, es decir, la suma de sus matrices.

Tema 4

Atención multi-cabeza y coste computacional

Una sola cabeza de atención define una única noción de parecido entre posiciones. La atención multi-cabeza ejecuta varias en paralelo, cada una en un subespacio de dimensión d/h, y combina sus resultados con una proyección lineal. Este tema la define, demuestra que su número de parámetros no depende del número de cabezas, muestra que cada cabeza tiene una forma bilineal de rango acotado, cuenta las operaciones de coma flotante de la capa y estudia la memoria de la matriz de atención, que es el obstáculo principal para los contextos largos. Se siguen la §3.2.2 y la §4 de [11], las §3.4.5 y §3.5 de la tarea 1 de CS336 [4], las §11.5 y §11.6.2 de [12] y las §7.1.1 y §7.3 de [8].

4.1 Definición

Definición 4.1 (Atención multi-cabeza). Sean d la dimensión del modelo, h un divisor de d y $d_k = d_v = d/h$. Para cada cabeza $r = 1, \dots, h$ se toman matrices $W_Q^{(r)}, W_K^{(r)}, W_V^{(r)} \in \mathbb{R}^{d \times d/h}$, y además $W_O \in \mathbb{R}^{d \times d}$. La *atención multi-cabeza* de $X \in \mathbb{R}^{n \times d}$ es

$$\mathrm{MHA}(X) = \mathrm{Concat}\big(H_1, \dots, H_h\big)\, W_O, \qquad H_r = \mathrm{Attn}\big(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)}\big) \in \mathbb{R}^{n \times d/h},$$

donde Concat yuxtapone las h matrices por columnas y la atención de cada cabeza puede llevar máscara causal.

Vaswani *et al.* usan $d = 512$ y $h = 8$, de modo que $d_k = d_v = 64$, y observan que el coste total es similar al de una sola cabeza de dimensión completa [11, §3.2.2]. Zhang *et al.* dan la definición con dimensiones generales y la misma elección [12, §11.5.1].

Observación 4.2 (Implementación). No se calculan h productos pequeños. Se apilan las matrices de las cabezas en $W_Q = \big(W_Q^{(1)} \ \cdots \ W_Q^{(h)}\big) \in \mathbb{R}^{d \times d}$, se calcula $Q = XW_Q$ con un único producto y se reordena el resultado de forma (n, d) a $(h, n, d/h)$; la fórmula de la atención se aplica entonces con un eje adicional de cabezas, que se trata como un eje de lote (módulo 3). La tarea 1 de CS336 pide calcular así las tres proyecciones con tres productos de matrices, o con uno solo concatenando las tres matrices [4, §3.4.5].

4.2 Parámetros y estructura

Proposición 4.3 (Parámetros). *Sin sesgos, la atención multi-cabeza tiene $4d^2$ parámetros, cualquiera que sea h.*

Con $d = 768$, la capa tiene $4 \cdot 768^2 = 2\,359\,296$ parámetros; con $d = 512$, $1\,048\,576$. En el módulo 14 se suman los $8d^2$ de la red de alimentación hacia delante con $d_{ff} = 4d$ y se obtiene que la atención es un tercio de los parámetros de cada bloque.

Proposición 4.4 (Descomposición por cabezas). *Sea $W_O^{(r)} \in \mathbb{R}^{d/h \times d}$ el bloque de filas $(r-1)d/h + 1, \ldots, rd/h$ de W_O. Entonces*

$$\mathrm{MHA}(X) = \sum_{r=1}^{h} H_r \, W_O^{(r)} = \sum_{r=1}^{h} A_r \, X \, W_V^{(r)} W_O^{(r)},$$

donde A_r es la matriz de atención de la cabeza r. Además, las puntuaciones de la cabeza r son $s_{ij}^{(r)} = x_i \, W_Q^{(r)} W_K^{(r)\top} x_j^\top$, una forma bilineal cuya matriz $W_{QK}^{(r)} = W_Q^{(r)} W_K^{(r)\top} \in \mathbb{R}^{d \times d}$ tiene rango a lo sumo d/h.

La proposición muestra que la multi-cabeza no es una sola atención grande cortada en trozos: es una *suma* de h atenciones, cada una con su propia matriz de pesos A_r, una forma bilineal de rango $\leq d/h$ que decide a dónde mirar y una aplicación $W_V^{(r)} W_O^{(r)}$ que decide qué escribir. Esta lectura es la base del análisis de circuitos del tema 5. La tarea 3 de CS224N ilustra con un ejemplo de claves ruidosas por qué promediar dos cabezas puede ser más robusto que una sola [3, tarea 3, pregunta 1].

Observación 4.5 (Variantes). Muchos modelos abiertos comparten las claves y los valores entre grupos de cabezas (atención *multi-query* y *grouped-query*) para reducir la memoria de inferencia; se estudian en el módulo 18. Qué variante usan Claude, Gemini o Grok en sus versiones actuales no se ha publicado en detalle.

4.3 Coste en operaciones

Se cuenta en operaciones de coma flotante (FLOP), con el convenio de CS336 de que una multiplicación seguida de una suma cuenta como dos [4, §3.5].

Lema 4.6. *El producto de una matriz $a \times b$ por una $b \times c$ requiere $2abc$ FLOP.*

Proposición 4.7 (Coste de la capa). *Con h cabezas de dimensión d/h, una pasada de la atención multi-cabeza sobre n posiciones requiere*

$$\begin{aligned} \mathrm{FLOP}_{proy} &= 8nd^2 && \text{(proyecciones } Q, K, V \text{ y } W_O\text{)}, \\ \mathrm{FLOP}_{at} &= 4n^2 d && \text{(productos } QK^\top \text{ y } AV\text{)}, \end{aligned}$$

sin contar la softmax, que es de orden hn^2. La parte cuadrática domina si y solo si $n > 2d$.

Con $d = 512$, la atención propiamente dicha domina a partir de 1024 tokens. Vaswani *et al.* resumen esta comparación en su tabla 1: complejidad por capa $O(n^2 d)$ para la autoatención frente a $O(nd^2)$ para una red recurrente, a cambio de $O(1)$ operaciones secuenciales y longitud máxima de camino $O(1)$ entre dos posiciones [11, §4]. Zhang *et al.* hacen el mismo análisis en [12, §11.6.2].

4.4 Memoria y contexto largo

La matriz de atención tiene n^2 entradas por cabeza y por capa. En fp16, con dos bytes por número (módulo 11), ocupa $2n^2$ bytes.

n (tokens)	Memoria de A (una cabeza, una capa, fp16)
4096	0,034 GB
10^5	20 GB
10^6	2000 GB

Cuadro 4.1: Memoria de una matriz de atención completa (1 GB = 10^9 bytes).

Multiplicar el contexto por k multiplica esta memoria y los $4n^2d$ FLOP por k^2. Las técnicas que lo mitigan se estudian en módulos posteriores: FlashAttention [5] calcula la softmax por bloques sin almacenar A completa, de modo que la memoria adicional pasa a ser lineal en n mientras el número de FLOP sigue siendo $O(n^2d)$ (módulo 17); las ventanas deslizantes y la atención dispersa reducen el conjunto permitido $\mathcal{P}$; y en inferencia la caché de claves y valores evita recalcular las del pasado (módulo 19).

Observación 4.8 (Contexto largo en modelos comerciales). El informe técnico de Gemini 1.5 [7] documenta recuperación casi perfecta en experimentos con contextos de hasta diez millones de tokens, y Anthropic ofrece para Claude ventanas de cientos de miles de tokens. Ninguna de las dos empresas ha publicado cómo resuelven internamente el coste cuadrático.

Tema 5

Circuitos de atención y memorias asociativas

Los temas anteriores tratan la atención como una operación aislada. Este tema estudia dos lecturas estructurales de ella. La primera, debida a Elhage *et al.* [6], descompone cada cabeza en un circuito que decide a dónde mirar y otro que decide qué copiar, y muestra que dos cabezas en capas sucesivas pueden componerse para implementar una *cabeza de inducción*, el mecanismo que Olsson *et al.* [9] relacionaron con el aprendizaje en contexto. La segunda, debida a Ramsauer *et al.* [10], identifica la atención con la regla de actualización de una red de Hopfield moderna, una memoria asociativa direccionable por contenido. Se siguen esos tres artículos y la §10.6 de [8].

5.1 El flujo residual y los circuitos QK y OV

En un Transformer, cada capa *suma* su salida a la representación de cada posición, que recorre así la red como un *flujo residual* (módulos 11 y 14). Elhage *et al.* estudian modelos formados solo por capas de atención, en los que el vector final de la posición i es su embedding más las contribuciones de todas las cabezas de todas las capas [6]. Por la proposición 4.4, la contribución de una cabeza a la posición i es

$$\sum_{j \le i} \alpha_{ij}\, x_j\, W_{OV}, \qquad \alpha_{ij} = \operatorname{softmax}_j\!\Big(\frac{x_i\, W_{QK}\, x_j^{\top}}{\sqrt{d_k}}\Big),$$

con $W_{QK} = W_Q W_K^{\top}$ y $W_{OV} = W_V W_O^{(r)}$, ambas de tamaño $d \times d$ (en esta sección se omite el índice de la cabeza).

Definición 5.1 (Circuitos QK y OV). En una cabeza de atención, el *circuito QK* es la forma bilineal $W_{QK} = W_Q W_K^{\top}$, que determina los pesos de atención, y el *circuito OV* es la aplicación lineal $W_{OV} = W_V W_O$, que determina qué se escribe en el flujo residual de la posición que atiende a partir de la posición atendida.

Los dos circuitos son independientes: el primero decide *a dónde* se mira; el segundo, *qué* se copia. Si los vectores de entrada son *one-hot*, $x_i W_{QK} x_j^{\top}$ es una entrada de la matriz W_{QK} y la tabla completa de preferencias de la cabeza puede leerse directamente.

5.2 Cabezas de inducción

Una *cabeza de inducción* implementa la regla «si antes apareció a seguido de b y ahora vuelve a aparecer a, predecir b», es decir, completa patrones del tipo $[a][b]\ldots[a] \to [b]$ [9]. Permite

copiar del propio contexto secuencias que nunca se vieron en el entrenamiento, como un nombre propio inventado que se repite en un texto.

Proposición 5.2. *En un modelo de una sola capa de atención cuyas entradas en cada posición son el embedding de su token más el de su posición, la puntuación s_{ij} es función de (x_i, x_j). En consecuencia, para dos posiciones permitidas $j, l \le i$, el cociente α_{ij}/α_{il} solo depende de x_i, x_j y x_l; en particular, ninguna cabeza de esa capa puede preferir la posición j a la l en función del token* anterior *x_{j-1}.*

Hace falta, por tanto, que una capa anterior haya escrito en la posición j información sobre x_{j-1}. La construcción de Elhage *et al.* usa dos cabezas [6]:

1. una *cabeza de token anterior* en la capa 1, cuyo mapa de atención es la subdiagonal: la posición j atiende a $j-1$ y su circuito OV copia la identidad del token, de modo que escribe en j «mi token anterior era x_{j-1}»;
2. una *cabeza de inducción* en la capa 2, cuyas claves se construyen a partir de lo escrito por la capa 1 y cuya consulta es el token actual: la posición i atiende a las j con $x_{j-1} = x_i$, y su circuito OV copia el token x_j.

Que las claves de la capa 2 lean la salida de una cabeza de la capa 1 se llama *composición K*; análogamente se definen las composiciones Q y V. La figura 5.1 ilustra el mecanismo.

Teorema 5.3 (Cabeza de token anterior). *Sean $p_0, \dots, p_{n-1}$ vectores* one-hot *de posición y $\beta > 0$. Con máscara causal, consultas $q_i = \beta p_i$ y claves $k_j = \beta p_{j+1}$ (con $p_n = 0$), y sin escalado adicional, el peso que la posición $i \ge 1$ asigna a $i-1$ es*

$$\alpha_{i,i-1} = \frac{e^{\beta^2}}{e^{\beta^2} + i} \xrightarrow[\beta\to\infty]{} 1.$$

Con valores $v_j = e_{x_j}$ (el *one-hot* del token), la salida de esta cabeza en la posición j es, para β grande, aproximadamente $e_{x_{j-1}}$. El ejercicio 5.4 completa la construcción con la segunda capa.

Observación 5.4 (Evidencia empírica). Olsson *et al.* presentan seis líneas de evidencia complementarias de que las cabezas de inducción explican una parte importante del aprendizaje en contexto: entre otras, que se forman durante el entrenamiento en el mismo intervalo en el que la capacidad de aprendizaje en contexto mejora bruscamente, visible como un escalón en la curva de pérdida. La evidencia es causal en modelos pequeños formados solo por atención y correlacional en modelos mayores con redes de alimentación hacia delante [9]. Jurafsky y Martin resumen estos resultados en la §10.6 de [8]; el módulo 23 retoma los circuitos.

5.3 Atención y redes de Hopfield modernas

Una memoria asociativa almacena patrones y, ante un estímulo parecido a uno de ellos, devuelve el patrón completo. Ramsauer *et al.* definen una red de Hopfield con estados continuos cuya regla de actualización es la atención [10, §2]. En esta sección se usa su convención: $X = (x_1, \dots, x_N) \in \mathbb{R}^{d \times N}$ tiene los patrones almacenados como *columnas* y el estado $\xi \in \mathbb{R}^d$ es un vector columna.

Definición 5.5 (Energía de Hopfield moderna). Sean $\beta > 0$ y $\operatorname{lse}(\beta, z) = \beta^{-1} \ln \sum_{l=1}^{N} e^{\beta z_l}$.

La energía del estado ξ es

$$E(\xi) = -\operatorname{lse}\big(\beta, X^\top \xi\big) + \tfrac{1}{2}\,\xi^\top \xi + \beta^{-1} \ln N + \tfrac{1}{2} M^2, \qquad M = \max_l \|x_l\|,$$

y la *regla de actualización* es $\xi^{\text{nuevo}} = X \operatorname{softmax}\big(\beta X^\top \xi\big)$.

Proposición 5.6 (Puntos estacionarios). *El gradiente de la energía es* $\nabla E(\xi) = \xi - X \operatorname{softmax}(\beta X^\top \xi)$. *Por tanto,* ξ *es un punto estacionario de* E *si y solo si es un punto fijo de la regla de actualización, y la actualización consiste en saltar al punto que anula el gradiente de la parte cuadrática con la parte* lse *congelada.*

Ramsauer *et al.* demuestran además que la actualización hace decrecer la energía y converge, que la red puede almacenar un número de patrones exponencial en la dimensión y que, si los patrones están bien separados, un solo paso basta para recuperar uno de ellos con error exponencialmente pequeño [10, §2, teoremas 1 a 5].

Observación 5.7 (Hopfield como atención). Si los patrones almacenados y los estados se proyectan linealmente para formar claves, valores y consultas y se toma $\beta = 1/\sqrt{d_k}$, la regla de actualización, escrita con vectores fila, adopta la forma $\operatorname{softmax}(QK^\top/\sqrt{d_k})V$ de la definición 2.1 [10, §2]. La atención es, pues, un paso de recuperación en una memoria asociativa, y β desempeña el papel de inverso de una temperatura: el teorema 1.5 describe sus dos regímenes extremos, recuperación de un patrón y media de todos ellos.

Bibliografía

[1] D. Bahdanau, K. Cho e Y. Bengio. Neural machine translation by jointly learning to align and translate. En *International Conference on Learning Representations (ICLR)*, 2015. https://arxiv.org/abs/1409.0473

[2] C. M. Bishop y H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. Cap. 12, «Transformers» (§12.1 «Attention»). https://www.bishopbook.com/

[3] D. Yang y Y. Choi. CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2026. Clase 5, «Transformers»; tarea 3, «Self-Attention and Transformers». https://web.stanford.edu/class/cs224n/

[4] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 1, «Basics», versión 26.0.3. Stanford University, primavera de 2026. https://github.com/stanford-cs336/assignment1-basics

[5] T. Dao, D. Y. Fu, S. Ermon, A. Rudra y C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. En *Advances in Neural Information Processing Systems 35*, 2022. https://arxiv.org/abs/2205.14135

[6] N. Elhage, N. Nanda, C. Olsson *et al.* A mathematical framework for Transformer circuits. *Transformer Circuits Thread*, Anthropic, 2021. https://transformer-circuits.pub/2021/framework/index.html

[7] Gemini Team, Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. Informe técnico, 2024. https://arxiv.org/abs/2403.05530

[8] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 7, «Transformers and Pretraining»; cap. 10, «Interpretability». https://web.stanford.edu/~jurafsky/slp3/

[9] C. Olsson, N. Elhage, N. Nanda *et al.* In-context learning and induction heads. *Transformer Circuits Thread*, Anthropic, 2022. https://arxiv.org/abs/2209.11895

[10] H. Ramsauer, B. Schäfl, J. Lehner, P. Seidl, M. Widrich, T. Adler, L. Gruber, M. Holzleitner, M. Pavlović, G. K. Sandve, V. Greiff, D. Kreil, M. Kopp, G. Klambauer, J. Brandstetter y S. Hochreiter. Hopfield networks is all you need. En *International Conference on Learning Representations (ICLR)*, 2021. https://arxiv.org/abs/2008.02217

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin. Attention is all you need. En *Advances in Neural Information Processing Systems 30*, 2017. https://arxiv.org/abs/1706.03762

[12] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. Cap. 11, «Attention Mechanisms and Transformers». https://d2l.ai/